# Lab 11: Linux Essentials

Brittany Rose

CITC 1332 Linux Operating System

Instructor: Dr. Noman Saied

04/08/2026

# Table of Contents


13.1 ..... 3

13.2 ..... 4

13.2.1 ..... 5

13.2.2 ..... 6

13.2.3 ..... 7

13.3 ..... 8

13.3.1 ..... 9

13.3.2 ..... 10

13.3.3 ..... 11

13.3.4 ..... 12

13.3.5 ..... 13

13.3.6 ..... 14

13.3.7 ..... 15

13.3.8 ..... 16

13.3.9 ..... 17

13.3.10 ..... 18

13.3.11 ..... 19

13.3.12 ..... 20

13.3.13 ..... 21

13.3.14 ..... 22

13.4 ..... 23

13.4.1 ..... 24

13.4.2 ..... 25

13.4.3 ..... 26

13.4.4 ..... 27

13.4.5 ..... 28

13.4.6 ..... 29

13.4.7 ..... 30

13.5 ..... 31

13.5.1 ..... 32

13.5.2 ..... 33

13.5.3 ..... 34

13.5.4 ..... 35

13.6 ..... 36

13.6.1 ..... 37

13.6.2 ..... 38

13.6.3 ..... 39

13.6.4 ..... 40

13.6.5 ..... 41

13.6.6 ..... 42

13.7 ..... 43

13.7.1 ..... 44

13.7.2 ..... 45

# 13.1 Introduction

This is **Lab 13: Where Data is Stored**. By performing this lab, students will learn about the locations of kernel information, process information, libraries, log files, and software packages.

In this lab, you will perform the following tasks:

- Investigate how the `/proc` filesystem is used by the kernel
- Use the `ps` command to view process information
- Learn how to manage processes by starting, stopping, and resuming them
- Viewing log files
- Manage the ability to load shared libraries

# 13.2 The kernel and /proc

In this task, you will explore the `/proc` directory and commands that communicate with the Linux kernel. The `/proc` directory appears to be an ordinary directory, like `/usr` or `/etc`, but it is not. Unlike the `/usr` or `/etc` directories, which are usually written to a disk drive, the `/proc` directory is a *pseudo filesystem* maintained in the memory of the computer.

**The /proc Directory**

The `/proc` directory contains a subdirectory for each running process on the system. Programs such as `ps` and `top` read the information about running processes from these directories. The `/proc` directory also contains information about the operating system and its hardware in files like `/proc/cpuinfo`, `/proc/meminfo` and `/proc/devices`.

The `/proc/sys` subdirectory contains *pseudo files* that can be used to alter the settings of the running kernel. Since these files are not "real" files, an editor should not be used to change them; instead you should use either the `echo` or the `sysctl` command to overwrite the contents of these files. For the same reason, do not attempt to view these files in an editor, but use the `cat` or `sysctl` command instead.

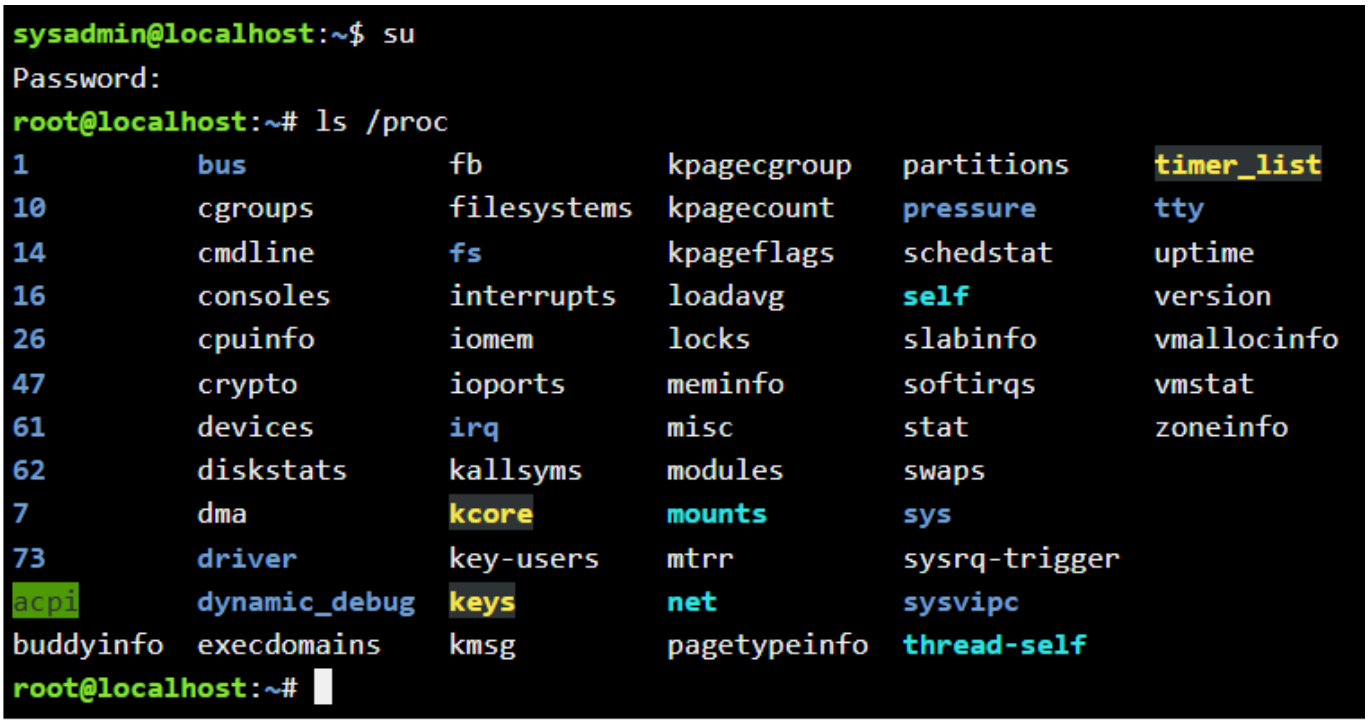
For permanent configuration changes, the kernel uses the `/etc/sysctl.conf` file. Typically, this file is used by the kernel to make changes to the /proc files when the system is starting up.

# 13.2.1 Step 1

In this task, you will examine some of the files contained in the `/proc` directory. Use the password `netlab123` when prompted:

```
su
ls /proc
```

Your output should be similar to the following:




Recall that the directories that have numbers for names represent running processes on the system. The first process is always `/sbin/init`, so the directory `/proc/1` will contain files with information about the running `init` process.

The `cmdline` file inside the process directory (`/proc/1/cmdline`, for example) will show the command that was executed. The order in which other processes are started varies greatly from system to system. Since the content of this file does not contain a new line character, an `echo` command will be executed to cause the prompt to go to a new line.

# 13.2.2 Step 2

Use `cat` then `ps` to view information about the `/sbin/init` process (Process IDentifier (PID) of 1):

```
cat /proc/1/cmdline; echo
ps -p 1
```

Your output should look something like this:

```
root@localhost:~# cat /proc/1/cmdline; echo
/sbin/init
root@localhost:~#
```

**Note**

The `echo` command in this example is executed immediately after the `cat` command. Since it has no argument, it functions only to put the next command prompt on a new line. Execute the `cat` command alone to see the difference.

```
root@localhost:~# ps -p 1
    PID TTY          TIME CMD
      1 pts/0    00:00:00 init
root@localhost:~#
```

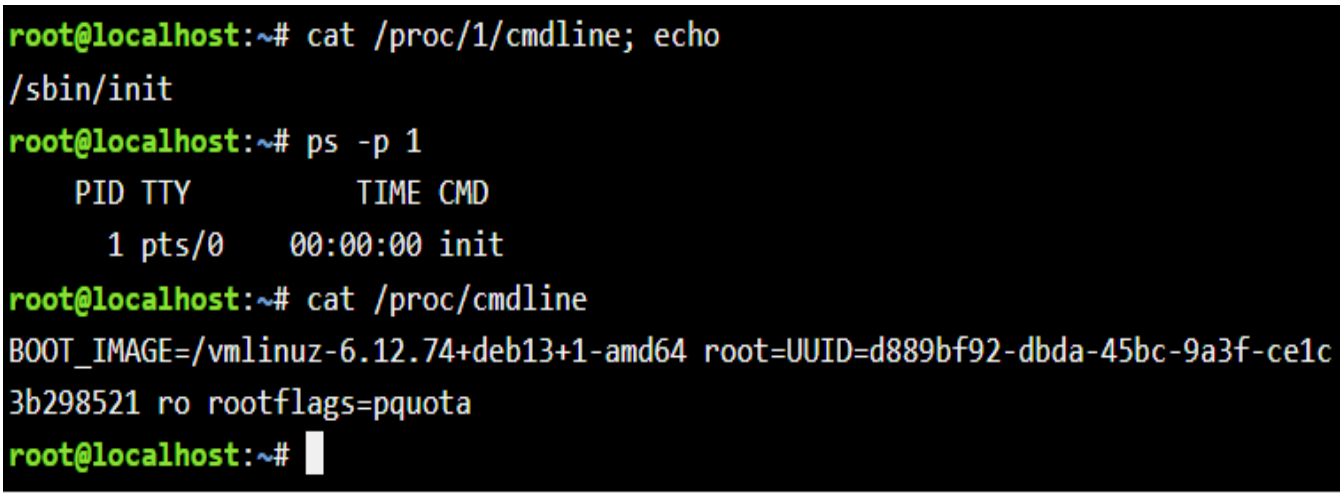
The other files in the /proc directory contain information about the operating system. The following tasks will be used to view and modify these files.

# 13.2.3 Step 3

View the `/proc/cmdline` file to see what arguments were passed to the kernel at boot time:

```
cat /proc/cmdline
```

The output of the command should be similar to this:




This output contains all of the information, such as command line parameters, special instructions, etc., that was passed to the kernel when it was first started.

# 13.3 Managing Processes

In this task you will start and stop processes.

>_ Ubuntu PC

```
sysadmin@localhost:~$ su
Password:
root@localhost:~# ls /proc
1           bus             fb           kpagecgroup   partitions     timer_list
10          cgroups         filesystems  kpagecount    pressure       tty
14          cmdline         fs           kpageflags    schedstat      uptime
16          consoles        interrupts   loadavg       self           version
26          cpuinfo         iomem        locks         slabinfo       vmallocinfo
47          crypto          ioports      meminfo       softirqs       vmstat
61          devices         irq          misc          stat           zoneinfo
62          diskstats       kallsyms     modules       swaps
7           dma             kcore        mounts        sys
73          driver          key-users    mtrr          sysrq-trigger
acpi        dynamic_debug   keys         net           sysvipc
buddyinfo   execdomains     kmsg         pagetypeinfo  thread-self
root@localhost:~# cat /proc/1/cmdline; echo
/sbin/init
root@localhost:~# ps -p 1
    PID TTY          TIME CMD
      1 pts/0    00:00:00 init
root@localhost:~# cat /proc/cmdline
BOOT_IMAGE=/vmlinuz-6.12.74+deb13+1-amd64 root=UUID=d889bf92-dbda-45bc-9a3f-ce1c
3b298521 ro rootflags=pquota
root@localhost:~#
```

# 13.3.1 Step 1

From the terminal, type the following command:

```
ping localhost > /dev/null
```

Your output should be similar to the following:

```
root@localhost:~# ping localhost > /dev/null
```

The output of the `ping` is being redirected to the `/dev/null` file (which is commonly known as the bit bucket).

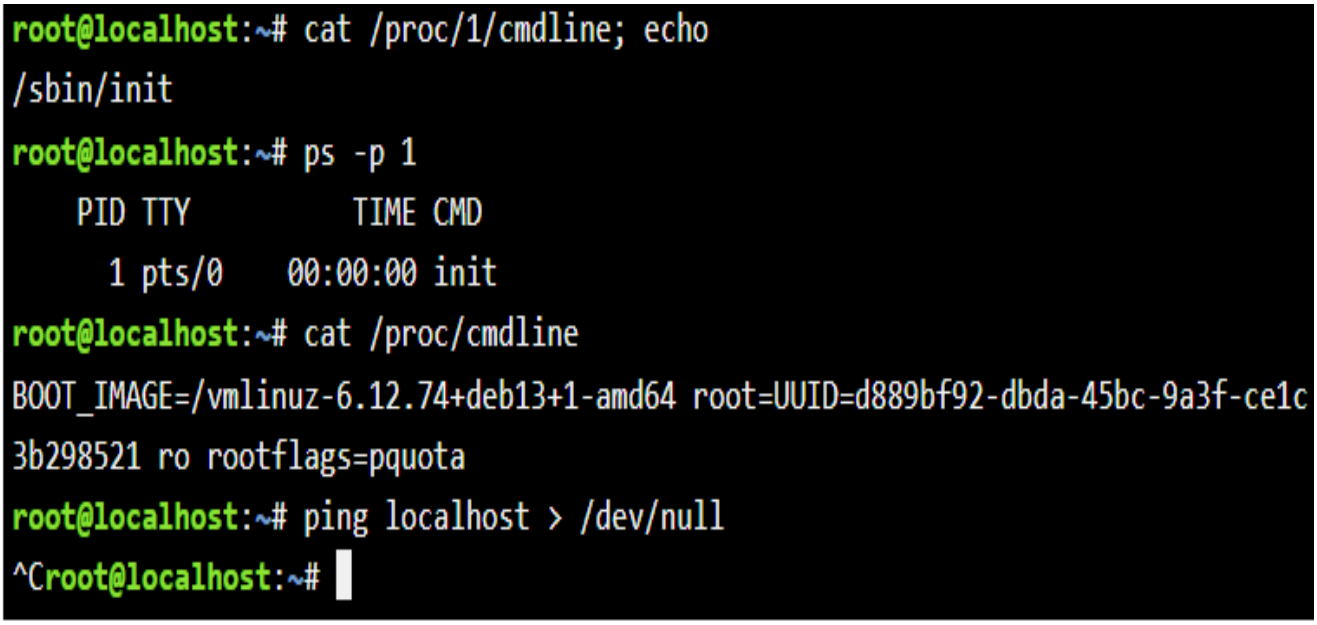
**Note**

Notice that the terminal appears to hang up at this command. This is due to running this command in the “foreground”. A process that is run in the foreground will prevent the user from using the shell until the process is complete. On the other hand, a process that is run in the background will allow the user to use the shell and execute other commands.

The system will continue to `ping` until the process is terminated or suspended by the user.

# 13.3.2 Step 2

Terminate the foreground process by pressing **Ctrl+C**.



# 13.3.3 Step 3

Next, to start the same process in the background, type:

```
ping localhost > /dev/null &
```

Your output should be similar to the following:

```
root@localhost:~# ping localhost > /dev/null &
[1] 78
```

By adding the ampersand `&` to the end of the command, the process is started in the background, allowing the user to maintain control of the terminal.

**Consider This**

An easier way to enter the above command would be to take advantage of the command history. You could have pressed the **Up Arrow Key** ↑ on the keyboard, added a **Space** and `&` to the end of the command and then pressed the **Enter** key. This is a time-saver when entering similar commands.

Notice that the previous command returns the following information:

```
[1] 78
```

This means that this process has a job number of 1 (as demonstrated by the output of `[1]`) and a Process ID (PID) of `78`. Each terminal/shell will have unique job numbers. The PID is system-wide; each process having a unique ID number.

This information is important when performing certain process manipulations, such as stopping processes or changing their priority value.

Your Process ID will likely be different than the one in the example.

# 13.3.4 Step 4

To see which commands are running in the current terminal, type the following command:

```
jobs
```

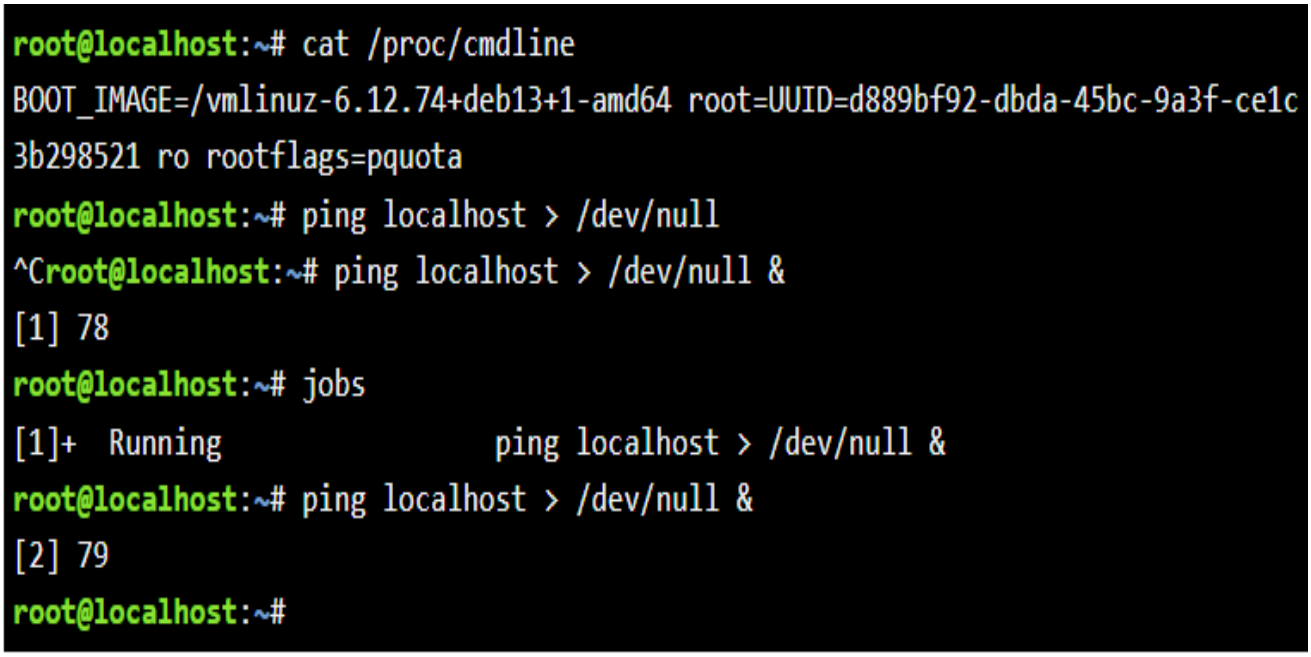
Your output should be similar to the following:

# 13.3.5 Step 5

Next, start another `ping` command in the background by typing the following:

```
ping localhost > /dev/null &
```

Your output should be similar to the following:




Notice the different job number and process ID for this new command.

# 13.3.6 Step 6

Now, there should be two `ping` commands running in the background. To verify, issue the `jobs` command again:

```
jobs
```

Your output should be similar to the following:

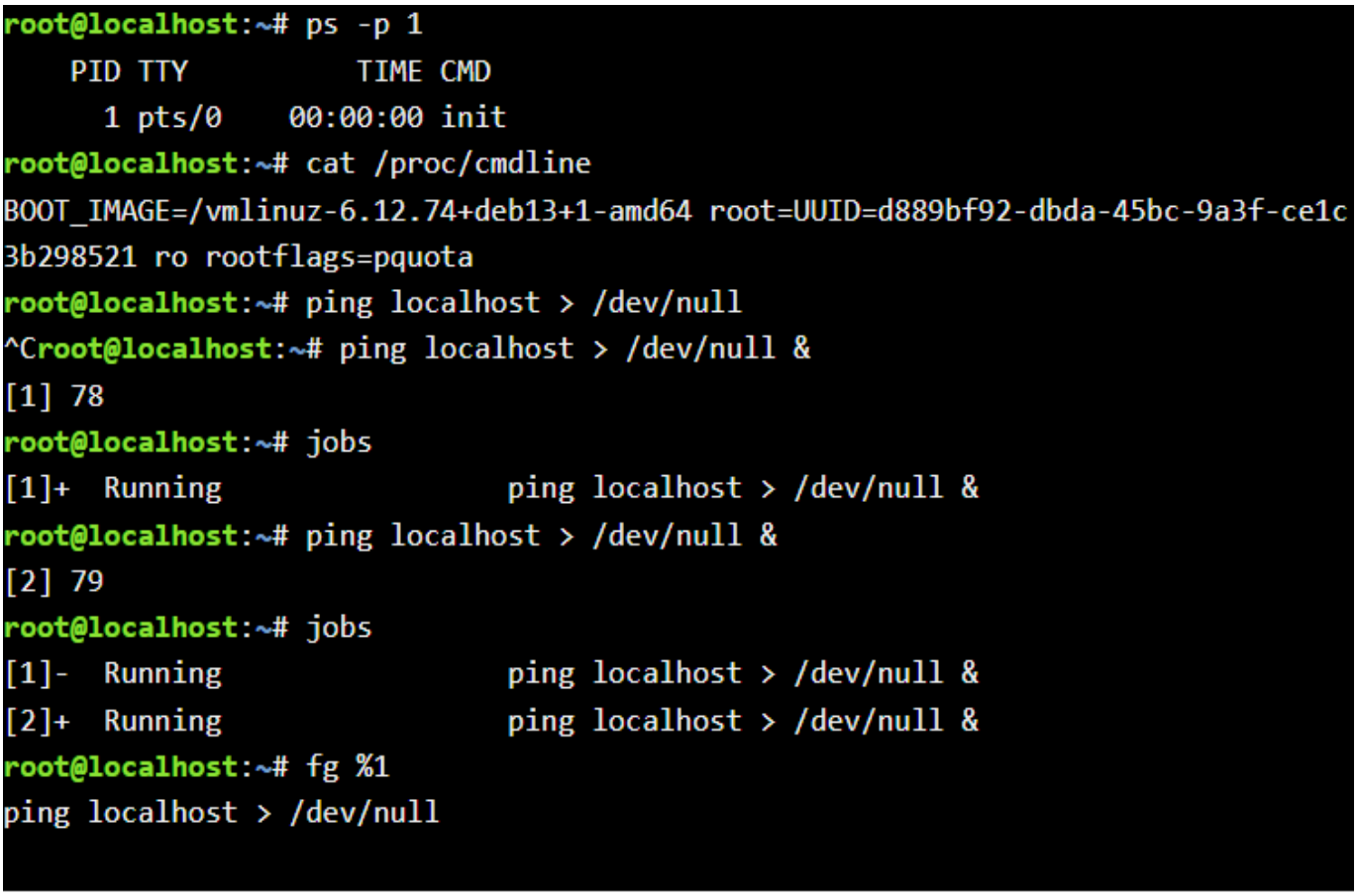
```
root@localhost:~# cat /proc/1/cmdline; echo
/sbin/init
root@localhost:~# ps -p 1
    PID TTY          TIME CMD
      1 pts/0    00:00:00 init
root@localhost:~# cat /proc/cmdline
BOOT_IMAGE=/vmlinuz-6.12.74+deb13+1-amd64 root=UUID=d889bf92-dbda-45bc-9a3f-ce1c
3b298521 ro rootflags=pquota
root@localhost:~# ping localhost > /dev/null
^Croot@localhost:~# ping localhost > /dev/null &
[1] 78
root@localhost:~# jobs
[1]+  Running                 ping localhost > /dev/null &
root@localhost:~# ping localhost > /dev/null &
[2] 79
root@localhost:~# jobs
[1]-  Running                 ping localhost > /dev/null &
[2]+  Running                 ping localhost > /dev/null &
root@localhost:~#
```

# 13.3.7 Step 7

Once you have verified that two `ping` commands are running, bring the first command to the foreground by typing the following:

```
fg %1
```

Your output should be similar to the following:



# 13.3.8 Step 8

Notice that, once again, the `ping` command has taken control of the terminal. To suspend (pause) the process and regain control of the terminal, type **Ctrl-Z**:

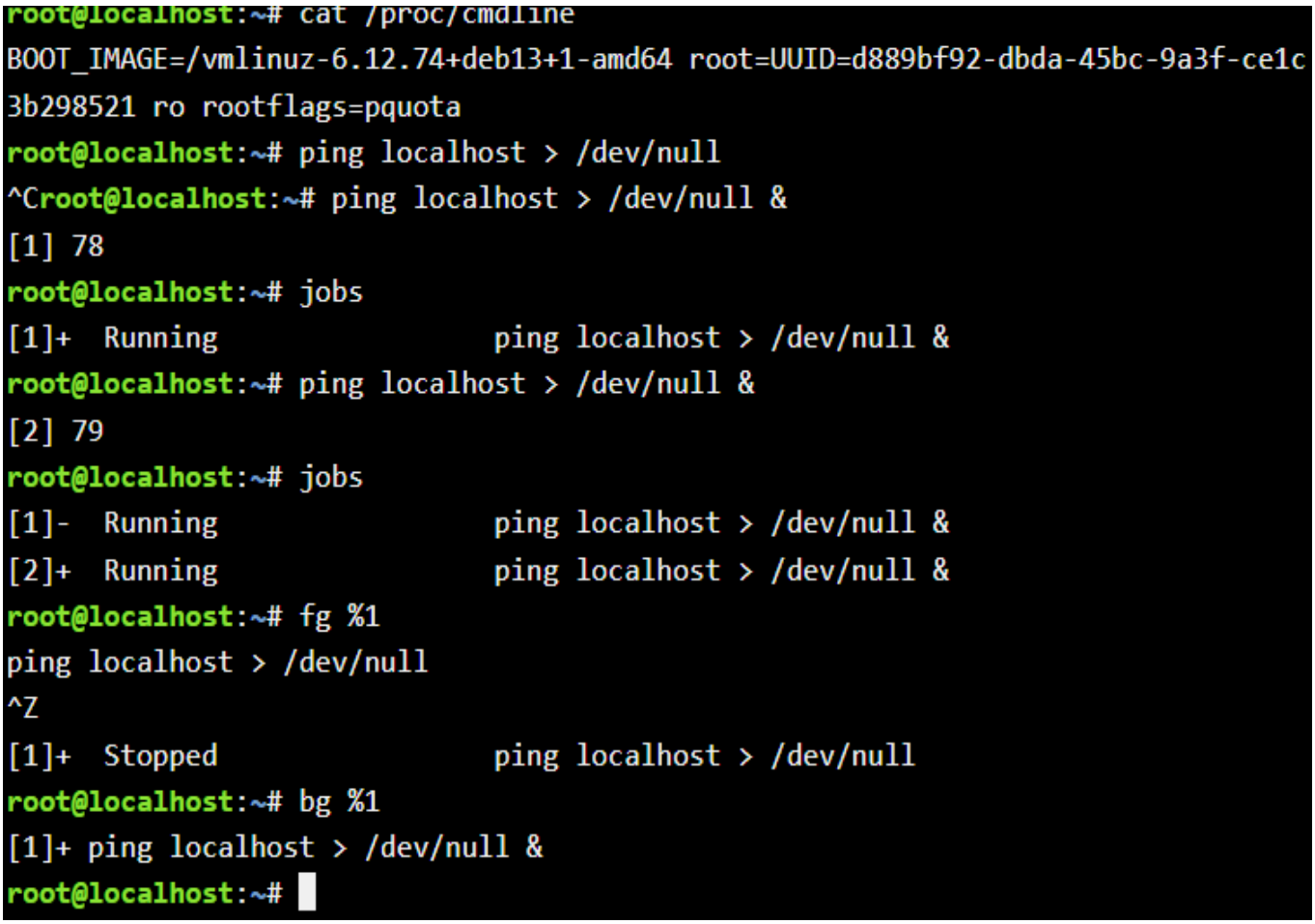
```
buddyinfo  execdomains    kmsg         pagetypeinfo  thread-self
root@localhost:~# cat /proc/1/cmdline; echo
/sbin/init
root@localhost:~# ps -p 1
    PID TTY          TIME CMD
      1 pts/0    00:00:00 init
root@localhost:~# cat /proc/cmdline
BOOT_IMAGE=/vmlinuz-6.12.74+deb13+1-amd64 root=UUID=d889bf92-dbda-45bc-9a3f-ce1c
3b298521 ro rootflags=pquota
root@localhost:~# ping localhost > /dev/null
^Croot@localhost:~# ping localhost > /dev/null &
[1] 78
root@localhost:~# jobs
[1]+  Running                 ping localhost > /dev/null &
root@localhost:~# ping localhost > /dev/null &
[2] 79
root@localhost:~# jobs
[1]-  Running                 ping localhost > /dev/null &
[2]+  Running                 ping localhost > /dev/null &
root@localhost:~# fg %1
ping localhost > /dev/null
^Z
[1]+  Stopped                 ping localhost > /dev/null
root@localhost:~#
```

# 13.3.9 Step 9

To have this process continue executing in the background, execute the following command:

```
bg %1
```

Your output should be similar to the following:



# 13.3.10 Step 10

Issue the `jobs` command again to verify two running processes:

```
jobs
```

Your output should be similar to the following:

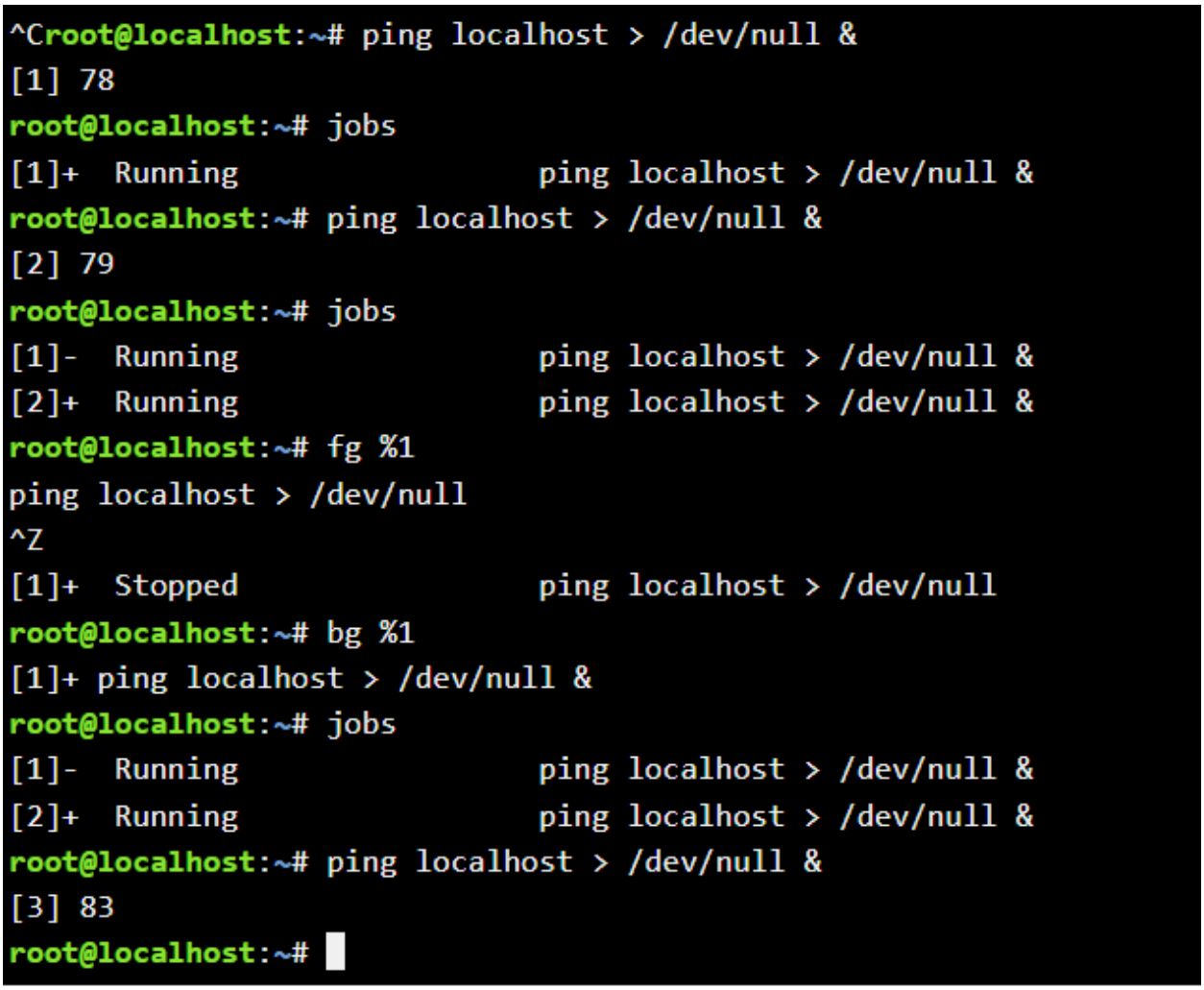
```
root@localhost:~# cat /proc/cmdline
BOOT_IMAGE=/vmlinuz-6.12.74+deb13+1-amd64 root=UUID=d889bf92-dbda-45bc-9a3f-ce1c
3b298521 ro rootflags=pquota
root@localhost:~# ping localhost > /dev/null
^Croot@localhost:~# ping localhost > /dev/null &
[1] 78
root@localhost:~# jobs
[1]+  Running                 ping localhost > /dev/null &
root@localhost:~# ping localhost > /dev/null &
[2] 79
root@localhost:~# jobs
[1]-  Running                 ping localhost > /dev/null &
[2]+  Running                 ping localhost > /dev/null &
root@localhost:~# fg %1
ping localhost > /dev/null
^Z
[1]+  Stopped                 ping localhost > /dev/null
root@localhost:~# bg %1
[1]+ ping localhost > /dev/null &
root@localhost:~# jobs
[1]-  Running                 ping localhost > /dev/null &
[2]+  Running                 ping localhost > /dev/null &
root@localhost:~#
```

# 13.3.11 Step 11

Next, start one more `ping` command by typing the following:

```
ping localhost > /dev/null &
```

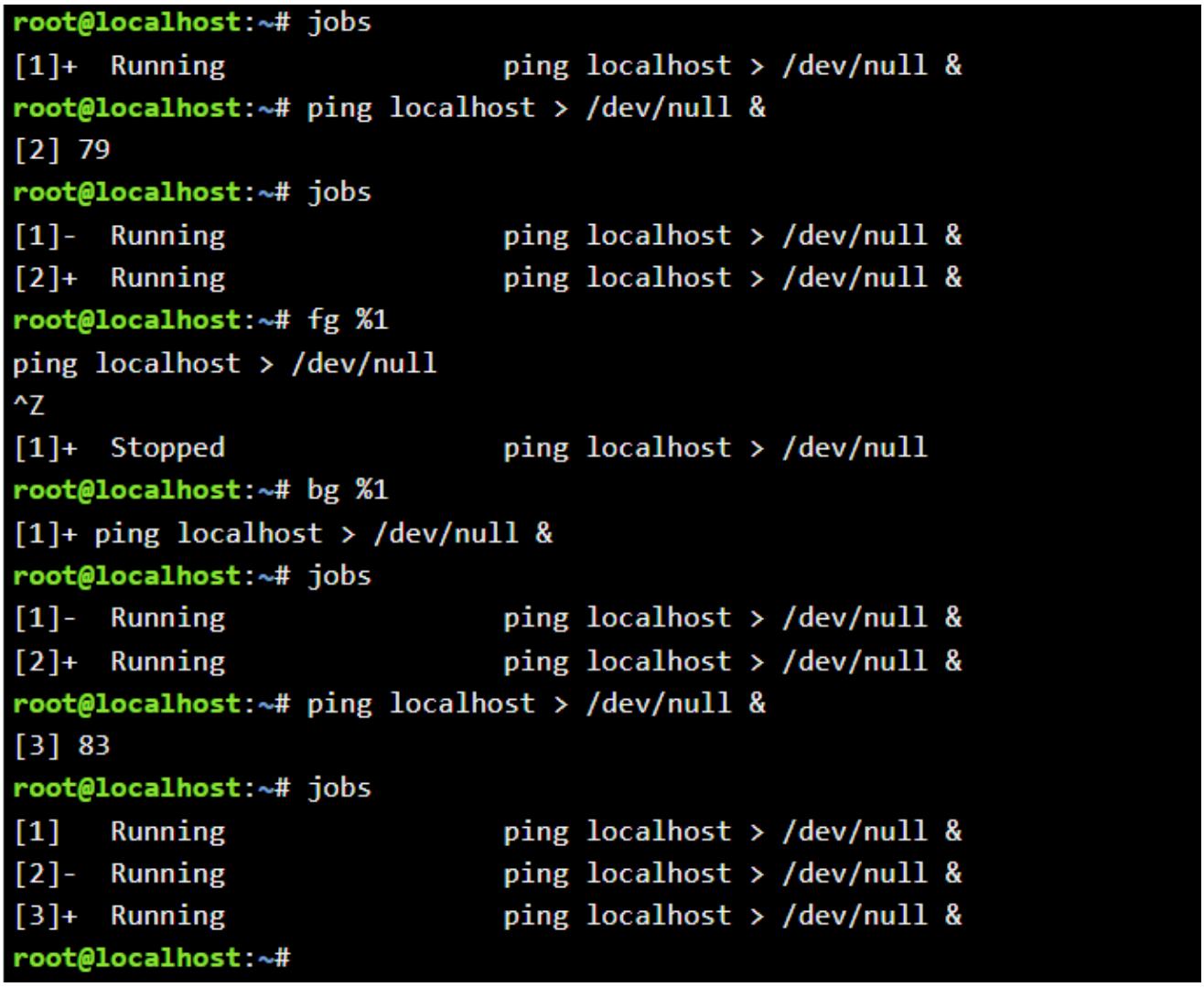
Your output should be similar to the following:



# 13.3.12 Step 12

Issue the `jobs` command again to verify three running processes:

```
jobs
```



# 13.3.13 Step 13

Using the job number, stop the last `ping` command with the `kill` command and verify it was stopped executing the `jobs` command:

```
kill %3
jobs
```

Your output should be similar to the following:

```
root@localhost:~# fg %1
ping localhost > /dev/null
^Z
[1]+  Stopped                 ping localhost > /dev/null
root@localhost:~# bg %1
[1]+ ping localhost > /dev/null &
root@localhost:~# jobs
[1]-  Running                 ping localhost > /dev/null &
[2]+  Running                 ping localhost > /dev/null &
root@localhost:~# ping localhost > /dev/null &
[3] 83
root@localhost:~# jobs
[1]   Running                 ping localhost > /dev/null &
[2]-  Running                 ping localhost > /dev/null &
[3]+  Running                 ping localhost > /dev/null &
root@localhost:~# kill %3
root@localhost:~# jobs
[1]   Running                 ping localhost > /dev/null &
[2]-  Running                 ping localhost > /dev/null &
[3]+  Terminated              ping localhost > /dev/null
root@localhost:~#
```

# 13.3.14 Step 14

Finally, you can stop all of the `ping` commands with the `killall` command. After executing the `killall` command, wait a few moments, and then run the `jobs` command to verify that all processes have stopped:

```
killall ping
jobs
```

Your output should be similar to the following:

```
root@localhost:~# jobs
[1]-  Running                 ping localhost > /dev/null &
[2]+  Running                 ping localhost > /dev/null &
root@localhost:~# ping localhost > /dev/null &
[3] 83
root@localhost:~# jobs
[1]   Running                 ping localhost > /dev/null &
[2]-  Running                 ping localhost > /dev/null &
[3]+  Running                 ping localhost > /dev/null &
root@localhost:~# kill %3
root@localhost:~# jobs
[1]   Running                 ping localhost > /dev/null &
[2]-  Running                 ping localhost > /dev/null &
[3]+  Terminated              ping localhost > /dev/null
root@localhost:~# killall ping
[1]-  Terminated              ping localhost > /dev/null
[2]+  Terminated              ping localhost > /dev/null
root@localhost:~# jobs
root@localhost:~#
```

# 13.4 Use Top to View Processes

In this task, you will use the `top` command to work with processes. By default, the `top` program sorts the processes in descending order of percentage of CPU usage, so the busiest programs will be at the top of its listing.

```
root@localhost:~# jobs
[1]-  Running                 ping localhost > /dev/null &
[2]+  Running                 ping localhost > /dev/null &
root@localhost:~# ping localhost > /dev/null &
[3] 83
root@localhost:~# jobs
[1]   Running                 ping localhost > /dev/null &
[2]-  Running                 ping localhost > /dev/null &
[3]+  Running                 ping localhost > /dev/null &
root@localhost:~# kill %3
root@localhost:~# jobs
[1]   Running                 ping localhost > /dev/null &
[2]-  Running                 ping localhost > /dev/null &
[3]+  Terminated              ping localhost > /dev/null
root@localhost:~# killall ping
[1]-  Terminated              ping localhost > /dev/null
[2]+  Terminated              ping localhost > /dev/null
root@localhost:~# jobs
root@localhost:~#
```

# 13.4.1 Step 1

From the terminal window, type the following commands:

```
ping localhost > /dev/null &
ping localhost > /dev/null &
```

Your output should be similar to the following:

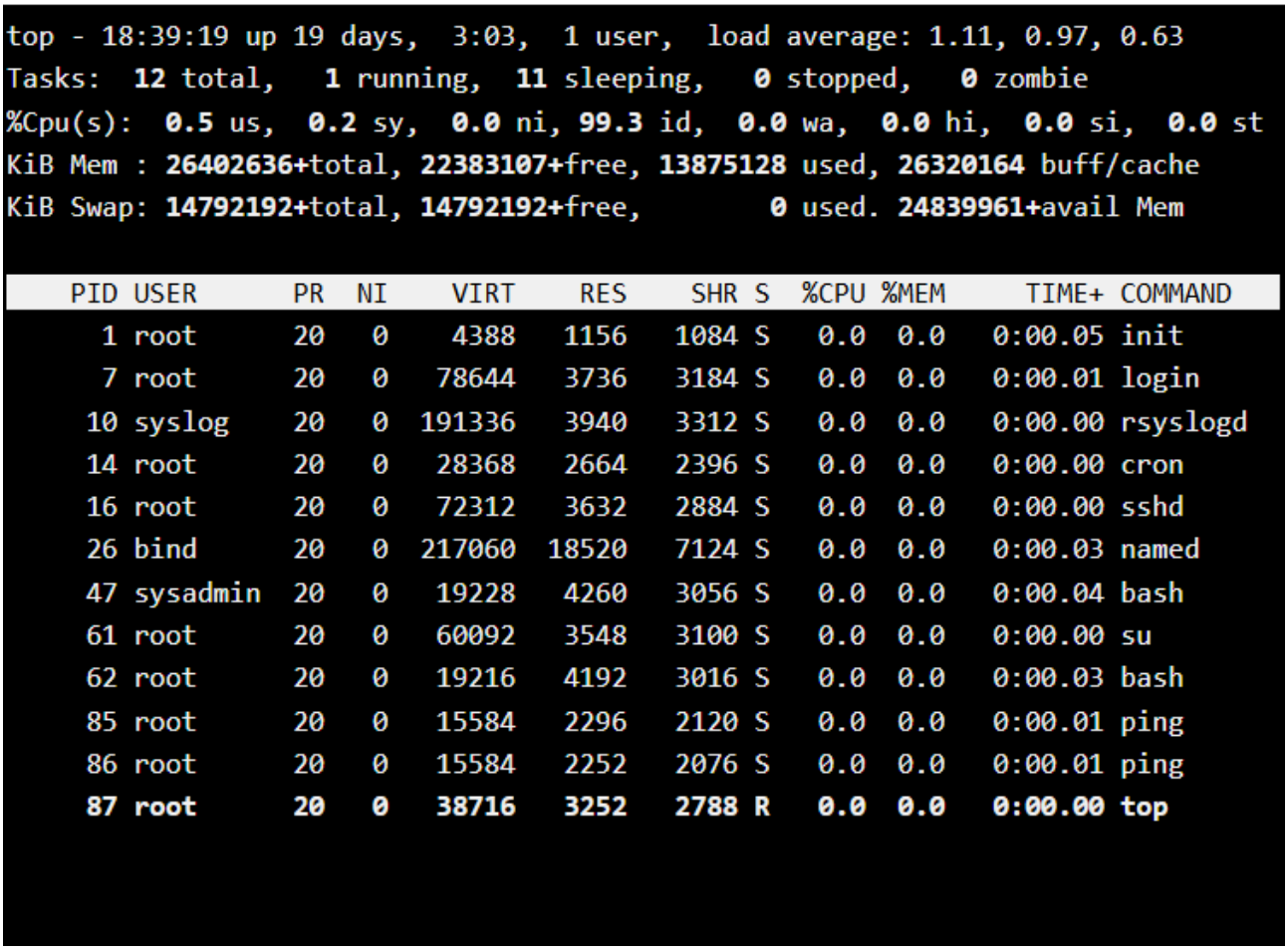
```
root@localhost:~# jobs
[1]   Running                 ping localhost > /dev/null &
[2]-  Running                 ping localhost > /dev/null &
[3]+  Running                 ping localhost > /dev/null &
root@localhost:~# kill %3
root@localhost:~# jobs
[1]   Running                 ping localhost > /dev/null &
[2]-  Running                 ping localhost > /dev/null &
[3]+  Terminated              ping localhost > /dev/null
root@localhost:~# killall ping
[1]-  Terminated              ping localhost > /dev/null
[2]+  Terminated              ping localhost > /dev/null
root@localhost:~# jobs
root@localhost:~# ping localhost > /dev/null &
[1] 85
root@localhost:~# ping localhost > /dev/null &
[2] 86
root@localhost:~#
```

Make note of the PIDs output by the above commands! They will be different than the examples that we are providing. You will use the PIDs in subsequent steps.

# 13.4.2 Step 2

Next, start the `top` command by typing the following in the terminal:

```
top
```

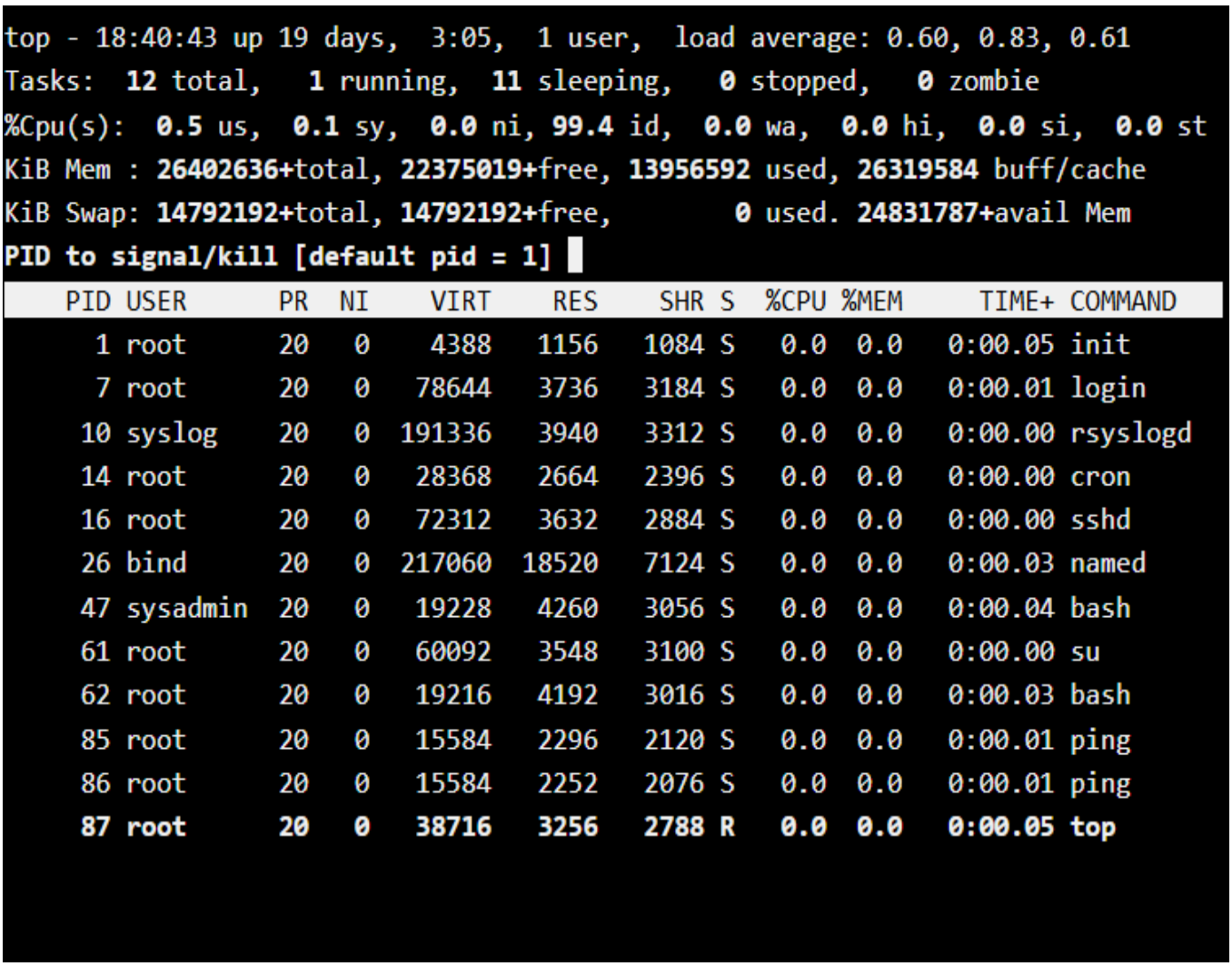



**Note**

The output of the `top` command changes every two seconds.

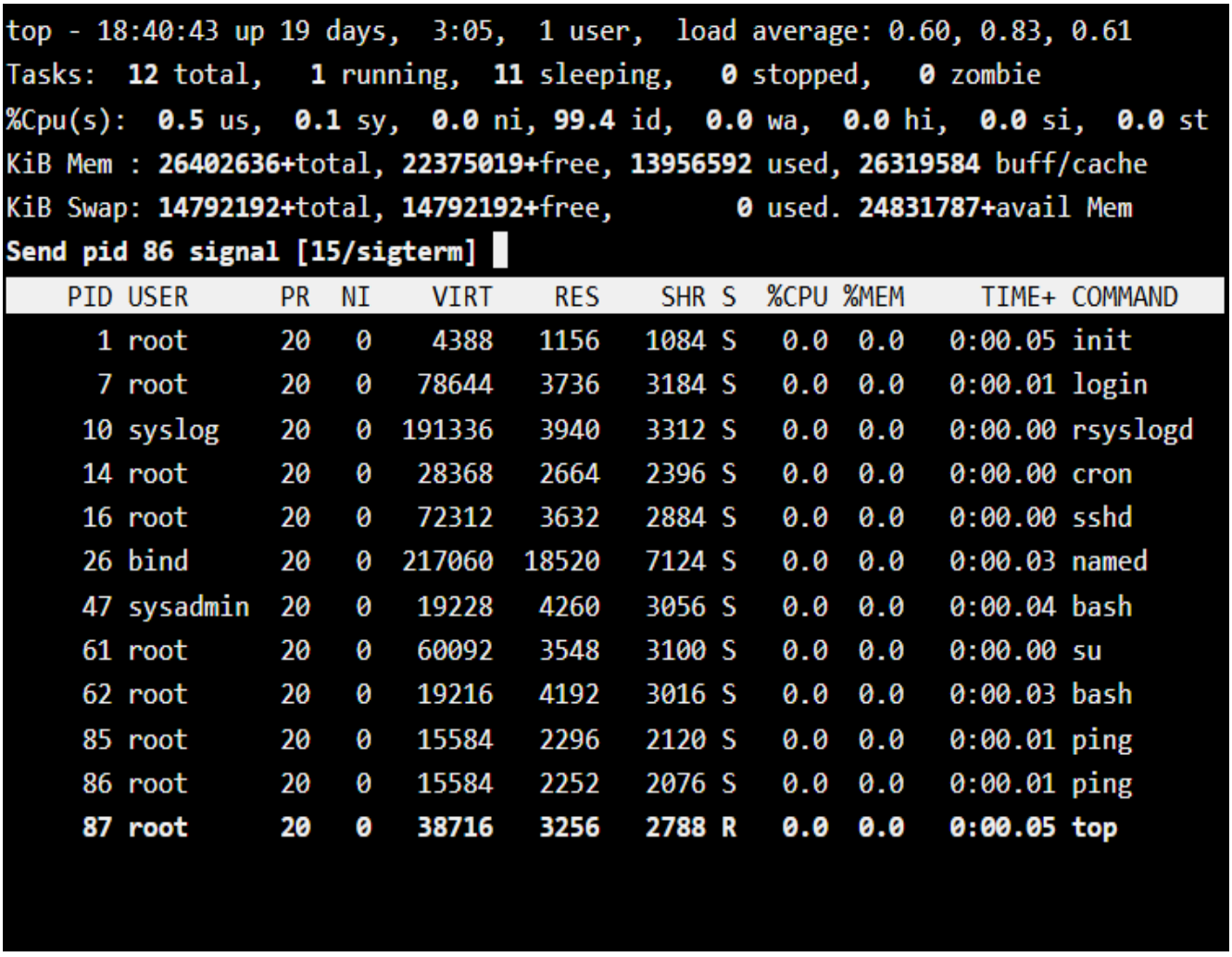
# 13.4.3 Step 3

The `top` command is an interactive program, which means that you can issue commands within the program. You will use the `top` command to kill the `ping` processes. First, type the letter **k**. Notice a prompt has appeared:



# 13.4.4 Step 4

At the `PID to kill:` prompt, type the PID of the first running `ping` process, then press **Enter**. Notice that the prompt changes as below:



# 13.4.5 Step 5

At the `Kill PID with signal [15]:` prompt, enter the signal to send to this process. In this case, just press the **Enter** key to use the default signal. Notice that the first `ping` command is removed and only one `ping` command remains in the listing (you may need to wait a few seconds as the `top` command refreshes):

```
top - 18:44:32 up 19 days,  3:08,  1 user,  load average: 1.17, 1.21, 0.82
Tasks:  11 total,   1 running,  10 sleeping,   0 stopped,   0 zombie
%Cpu(s):  1.5 us,  0.5 sy,  0.0 ni, 97.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem : 26402636+total, 22411331+free, 13659376 used, 26253672 buff/cache
KiB Swap: 14792192+total, 14792192+free,        0 used. 24861555+avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
    1 root      20   0    4388   1156   1084 S   0.0  0.0   0:00.05 init
    7 root      20   0   78644   3736   3184 S   0.0  0.0   0:00.01 login
   10 syslog    20   0  191336   3940   3312 S   0.0  0.0   0:00.00 rsyslogd
   14 root      20   0   28368   2664   2396 S   0.0  0.0   0:00.00 cron
   16 root      20   0   72312   3632   2884 S   0.0  0.0   0:00.00 sshd
   26 bind      20   0  217060  18520   7124 S   0.0  0.0   0:00.03 named
   47 sysadmin  20   0   19228   4260   3056 S   0.0  0.0   0:00.04 bash
   61 root      20   0   60092   3548   3100 S   0.0  0.0   0:00.00 su
   62 root      20   0   19216   4192   3016 S   0.0  0.0   0:00.03 bash
   85 root      20   0   15584   2296   2120 S   0.0  0.0   0:00.03 ping
   87 root      20   0   38716   3256   2788 R   0.0  0.0   0:00.07 top
```

**Consider This**

There are several different numeric values that can be sent to a process. These are predefined values, each with a different meaning. If you want to learn more about these values, type `man kill` in a terminal window.

The prompt indicates that the default signal is the terminate signal indicated by `SIGTERM` or the number `15`.

# 13.4.6 Step 6

Next, kill the remaining `ping` process as before, except this time, at the signal prompt `Kill PID with signal [15]:` prompt, use a value of `9` instead of accepting the default of `15`. Press **Enter** to accept the entry.

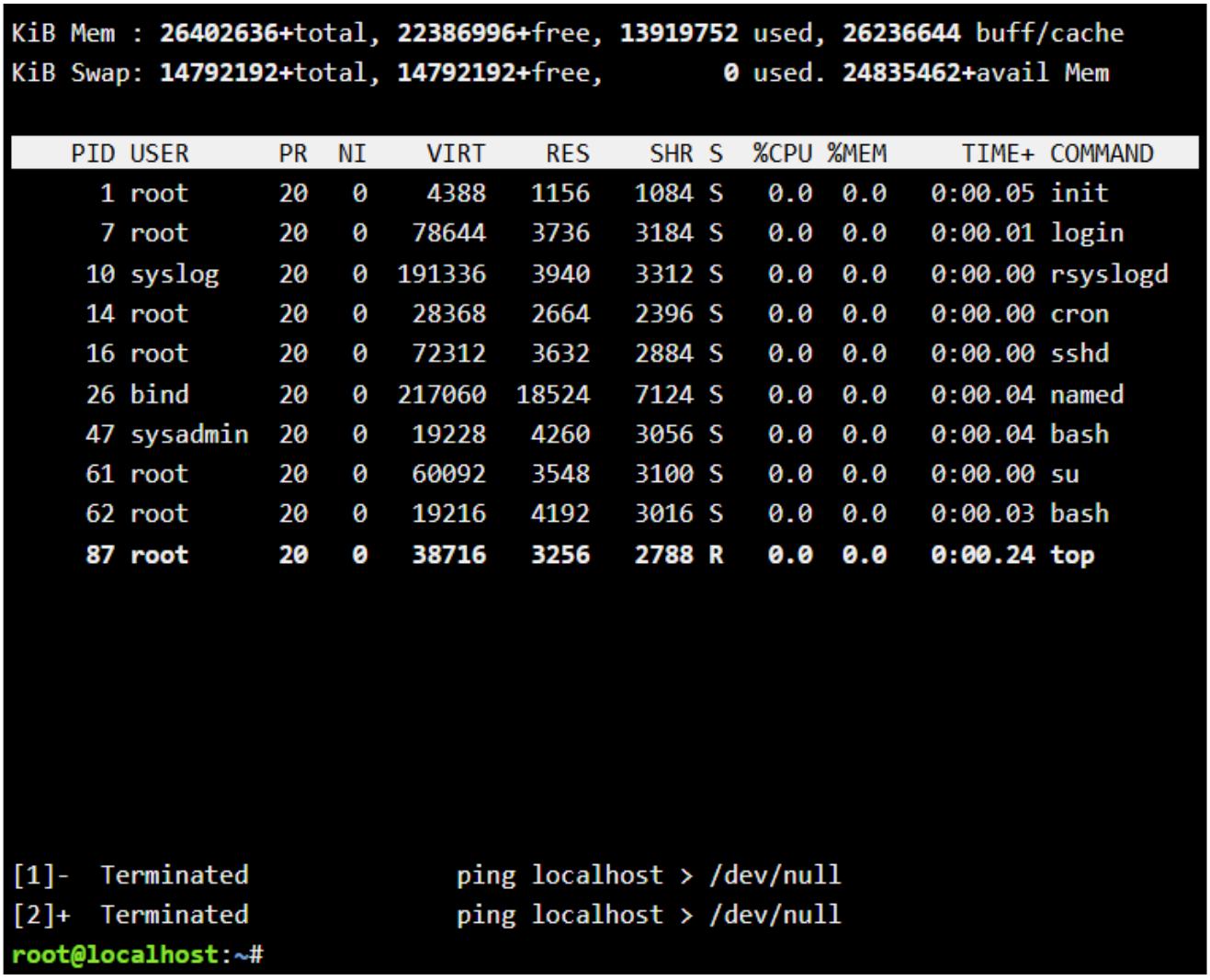
```
top - 18:48:32 up 19 days,  3:12,  1 user,  load average: 0.40, 0.69, 0.69
Tasks:  10 total,   1 running,   9 sleeping,   0 stopped,   0 zombie
%Cpu(s):  1.5 us,  0.5 sy,  0.0 ni, 98.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem : 26402636+total, 22393360+free, 13854044 used, 26238728 buff/cache
KiB Swap: 14792192+total, 14792192+free,        0 used. 24842041+avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
    1 root      20   0    4388   1156   1084 S   0.0  0.0   0:00.05 init
    7 root      20   0   78644   3736   3184 S   0.0  0.0   0:00.01 login
   10 syslog    20   0  191336   3940   3312 S   0.0  0.0   0:00.00 rsyslogd
   14 root      20   0   28368   2664   2396 S   0.0  0.0   0:00.00 cron
   16 root      20   0   72312   3632   2884 S   0.0  0.0   0:00.00 sshd
   26 bind      20   0  217060  18524   7124 S   0.0  0.0   0:00.04 named
   47 sysadmin  20   0   19228   4260   3056 S   0.0  0.0   0:00.04 bash
   61 root      20   0   60092   3548   3100 S   0.0  0.0   0:00.00 su
   62 root      20   0   19216   4192   3016 S   0.0  0.0   0:00.03 bash
   87 root      20   0   38716   3256   2788 R   0.0  0.0   0:00.22 top
```

**Consider This**

The kill signal `9` or `SIGKILL` is a "forceful" signal that cannot be ignored, unlike the default value of `15`. Notice that all references to the `ping` command are now removed from `top`.

# 13.4.7 Step 7

Type **q** to exit the `top` command. The following screen reflects that both `ping` commands were terminated:



## 13.5 Use pkill and kill To Terminate Processes

In this task, we will continue to work with processes. You will use `pkill` and `kill` to terminate processes.

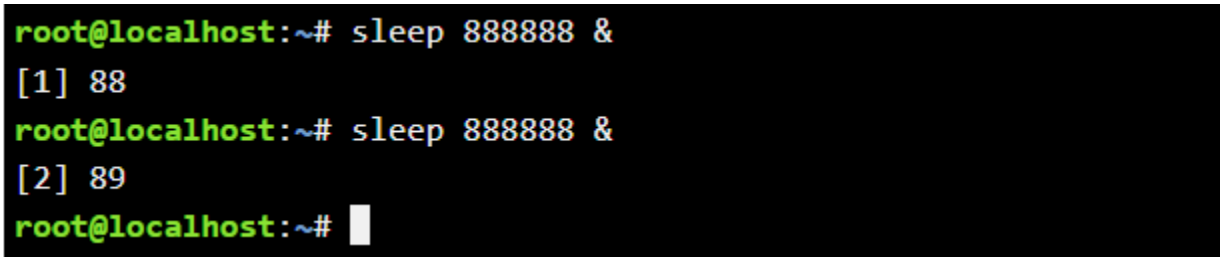
```
KiB Mem : 26402636+total, 22386996+free, 13919752 used, 26236644 buff/cache
KiB Swap: 14792192+total, 14792192+free,        0 used. 24835462+avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
    1 root      20   0    4388   1156   1084 S   0.0  0.0   0:00.05 init
    7 root      20   0   78644   3736   3184 S   0.0  0.0   0:00.01 login
   10 syslog    20   0  191336   3940   3312 S   0.0  0.0   0:00.00 rsyslogd
   14 root      20   0   28368   2664   2396 S   0.0  0.0   0:00.00 cron
   16 root      20   0   72312   3632   2884 S   0.0  0.0   0:00.00 sshd
   26 bind      20   0  217060  18524   7124 S   0.0  0.0   0:00.04 named
   47 sysadmin  20   0   19228   4260   3056 S   0.0  0.0   0:00.04 bash
   61 root      20   0   60092   3548   3100 S   0.0  0.0   0:00.00 su
   62 root      20   0   19216   4192   3016 S   0.0  0.0   0:00.03 bash
   87 root      20   0   38716   3256   2788 R   0.0  0.0   0:00.24 top



[1]-  Terminated              ping localhost > /dev/null
[2]+  Terminated              ping localhost > /dev/null
root@localhost:~#
```

# 13.5.1 Step 1

To begin, type the following commands in the terminal:

```
sleep 888888 &
sleep 888888 &
```

Your output should be similar to the following:




The `sleep` command is typically used to pause a program (shell script) for a specific period of time. In this case it is used just to provide a command that will take a long time to run.

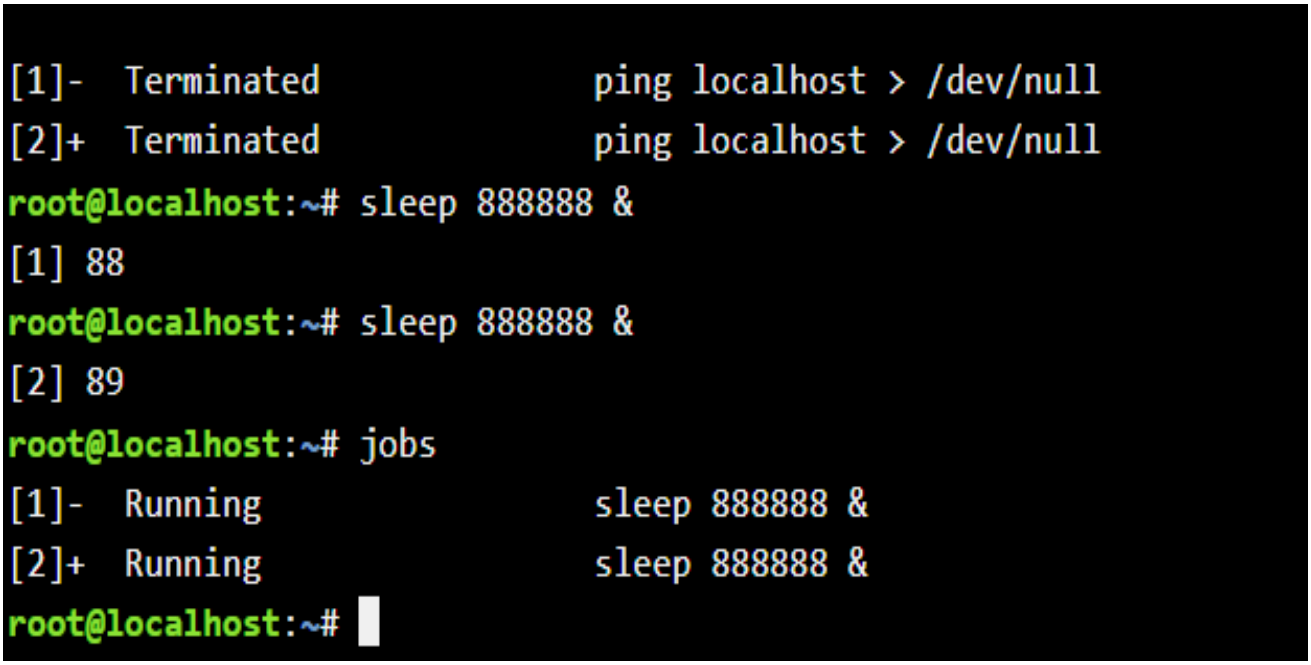
Make sure to note the PIDs of the `sleep` processes in your virtual machine for the next steps! Your PIDs will be different than those demonstrated in the lab.

# 13.5.2 Step 2

Next, determine which jobs are currently running by typing:

```
jobs
```

Your output should be similar to the following:



# 13.5.3 Step 3

Now, use the `kill` command to stop the first instance of the `sleep` command by typing the following (substitute *PID* with the process id of your first `sleep` command). Also, execute `jobs` to verify the process has been stopped:

```
ps
kill PID
jobs
```

Your output should be similar to the following:

```
root@localhost:~# ps
    PID TTY          TIME CMD
      1 pts/0    00:00:00 init
      7 pts/0    00:00:00 login
     61 pts/0    00:00:00 su
     62 pts/0    00:00:00 bash
     88 pts/0    00:00:00 sleep
     89 pts/0    00:00:00 sleep
     90 pts/0    00:00:00 ps
root@localhost:~# kill 88
root@localhost:~# jobs
[1]-  Terminated              sleep 888888
[2]+  Running                 sleep 888888 &
root@localhost:~#
```

**Helpful Hint**

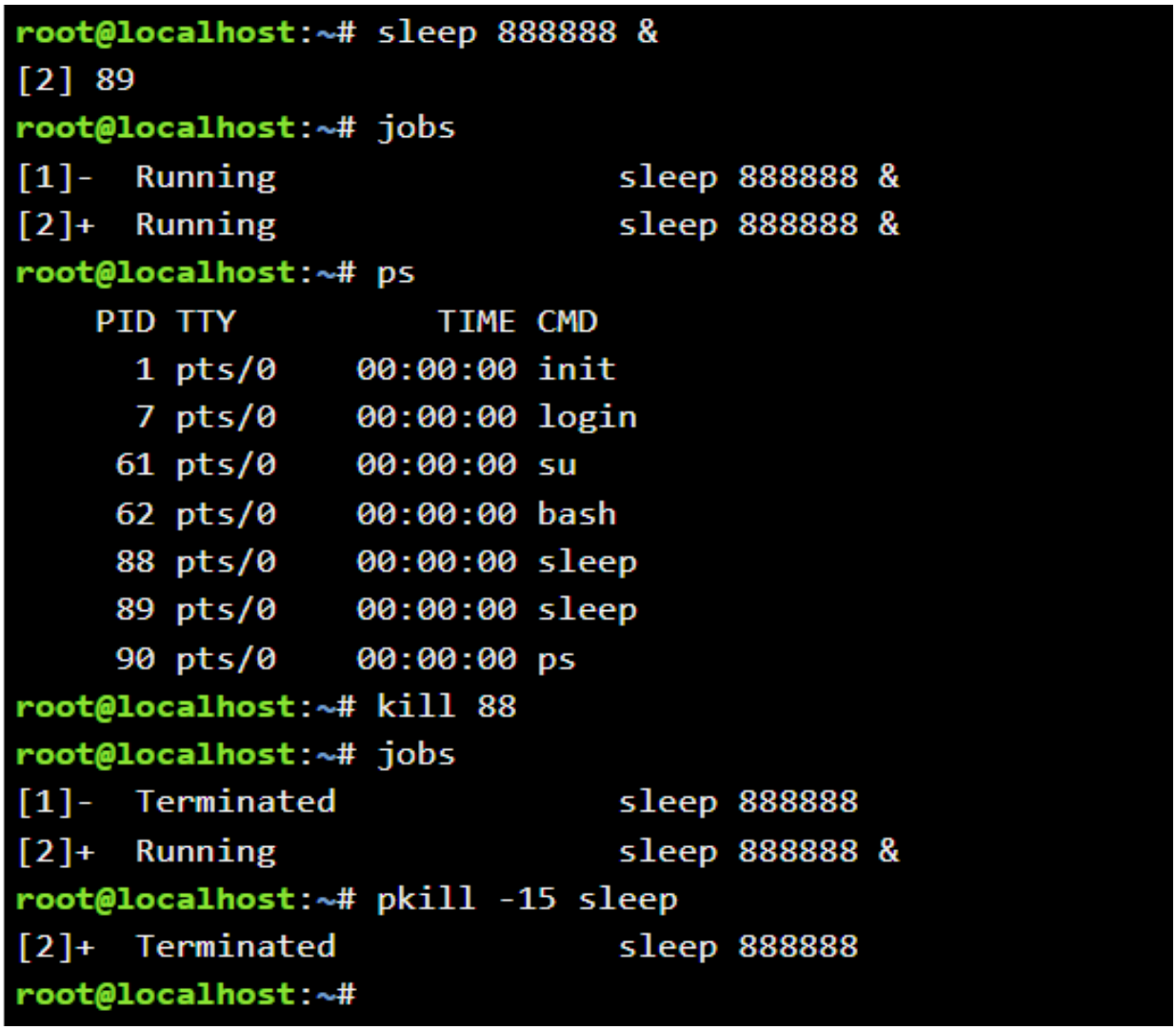
If you can't remember the PID of the first process, just type the `ps` command, as shown above.

# 13.5.4 Step 4

Next, use the `pkill` command to terminate the remaining `sleep` command, using the name of the program rather than the PID:

```
pkill -15 sleep
```

Your output should be similar to the following:



# 13.6 Using ps to Select and Sort Processes

The `ps` command can be used to view processes. By default, the `ps` command will only display the processes running in the current shell.

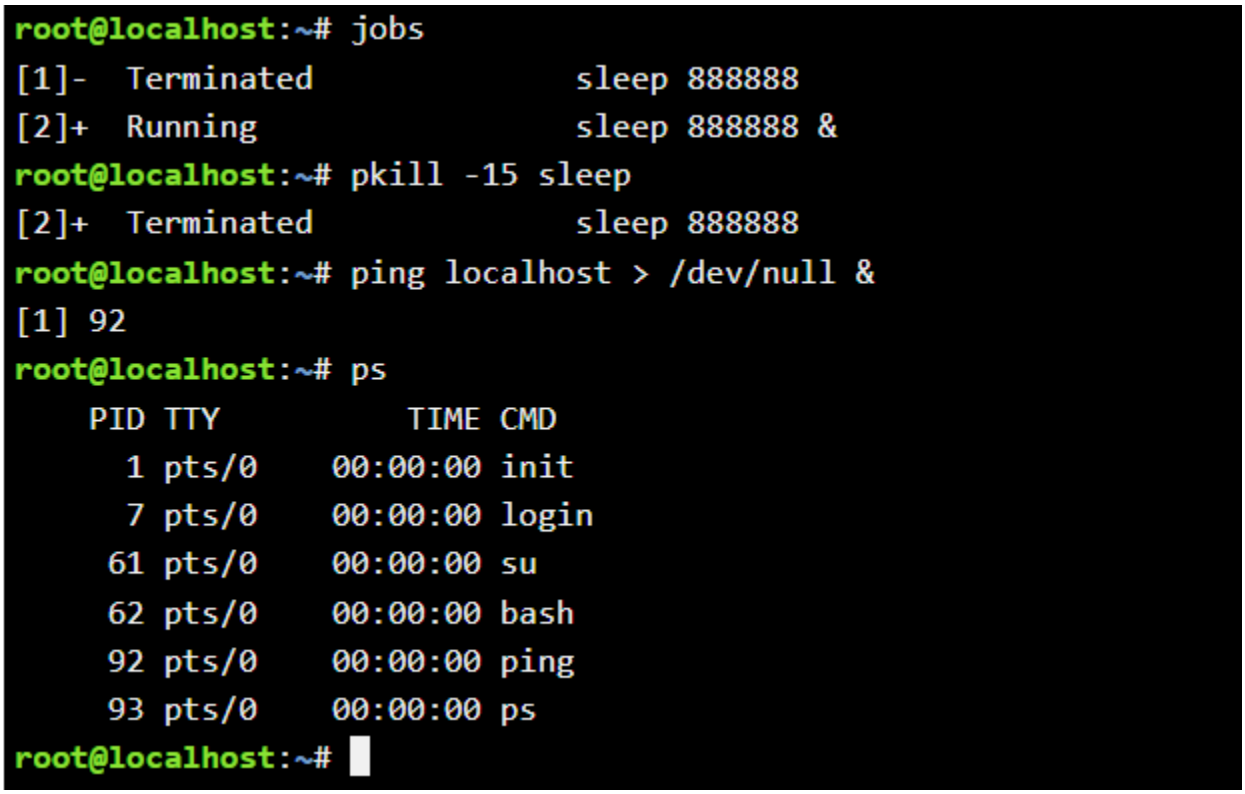
```
[2]+  Terminated              ping localhost > /dev/null
root@localhost:~# sleep 888888 &
[1] 88
root@localhost:~# sleep 888888 &
[2] 89
root@localhost:~# jobs
[1]-  Running                 sleep 888888 &
[2]+  Running                 sleep 888888 &
root@localhost:~# ps
    PID TTY          TIME CMD
      1 pts/0    00:00:00 init
      7 pts/0    00:00:00 login
     61 pts/0    00:00:00 su
     62 pts/0    00:00:00 bash
     88 pts/0    00:00:00 sleep
     89 pts/0    00:00:00 sleep
     90 pts/0    00:00:00 ps
root@localhost:~# kill 88
root@localhost:~# jobs
[1]-  Terminated              sleep 888888
[2]+  Running                 sleep 888888 &
root@localhost:~# pkill -15 sleep
[2]+  Terminated              sleep 888888
root@localhost:~#
```

# 13.6.1 Step 1

Start up a background process using `ping` and view current processes using the `ps` command:

```
ping localhost > /dev/null &
ps
```

Your output should be similar to the following:



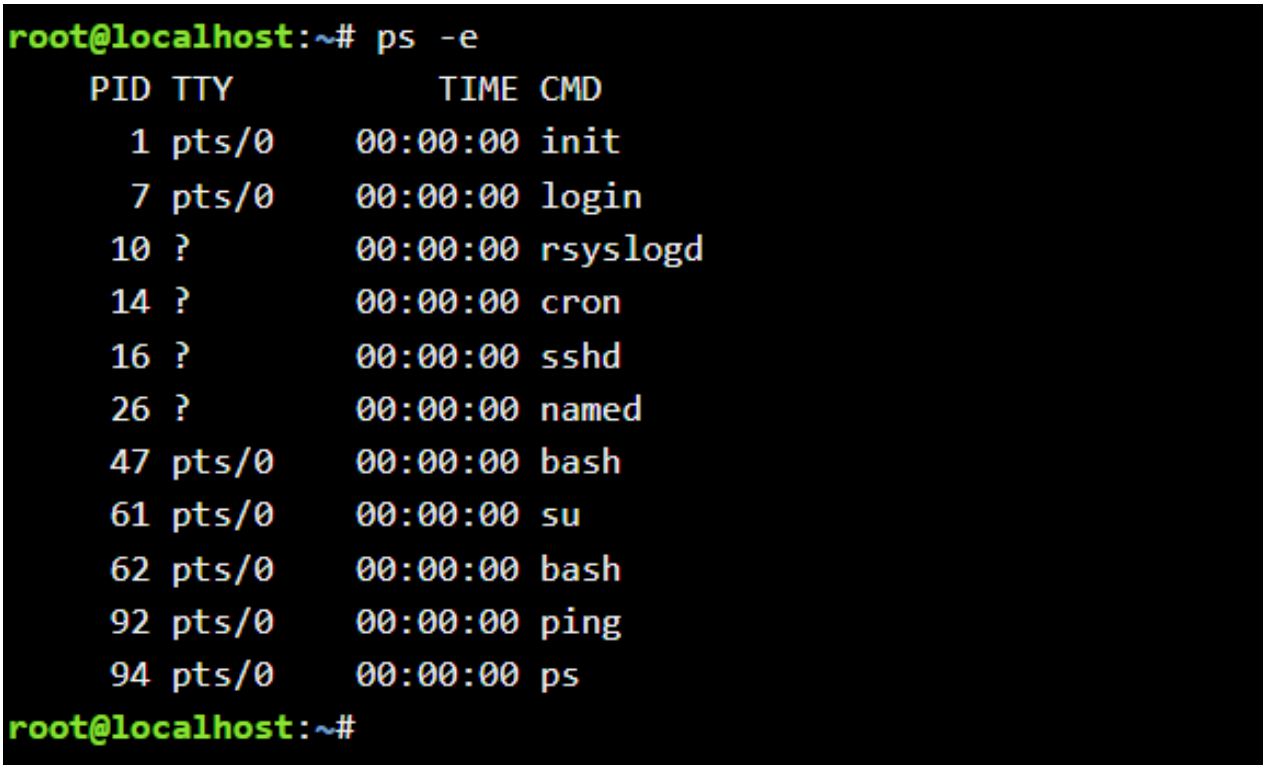

**Make note of the PID of the `ping`, you will use the PID in a subsequent step.**

# 13.6.2 Step 2

Execute the `ps` command using the option `-e`, so all processes are displayed.

```
ps -e
```

Your output should be similar to the following:



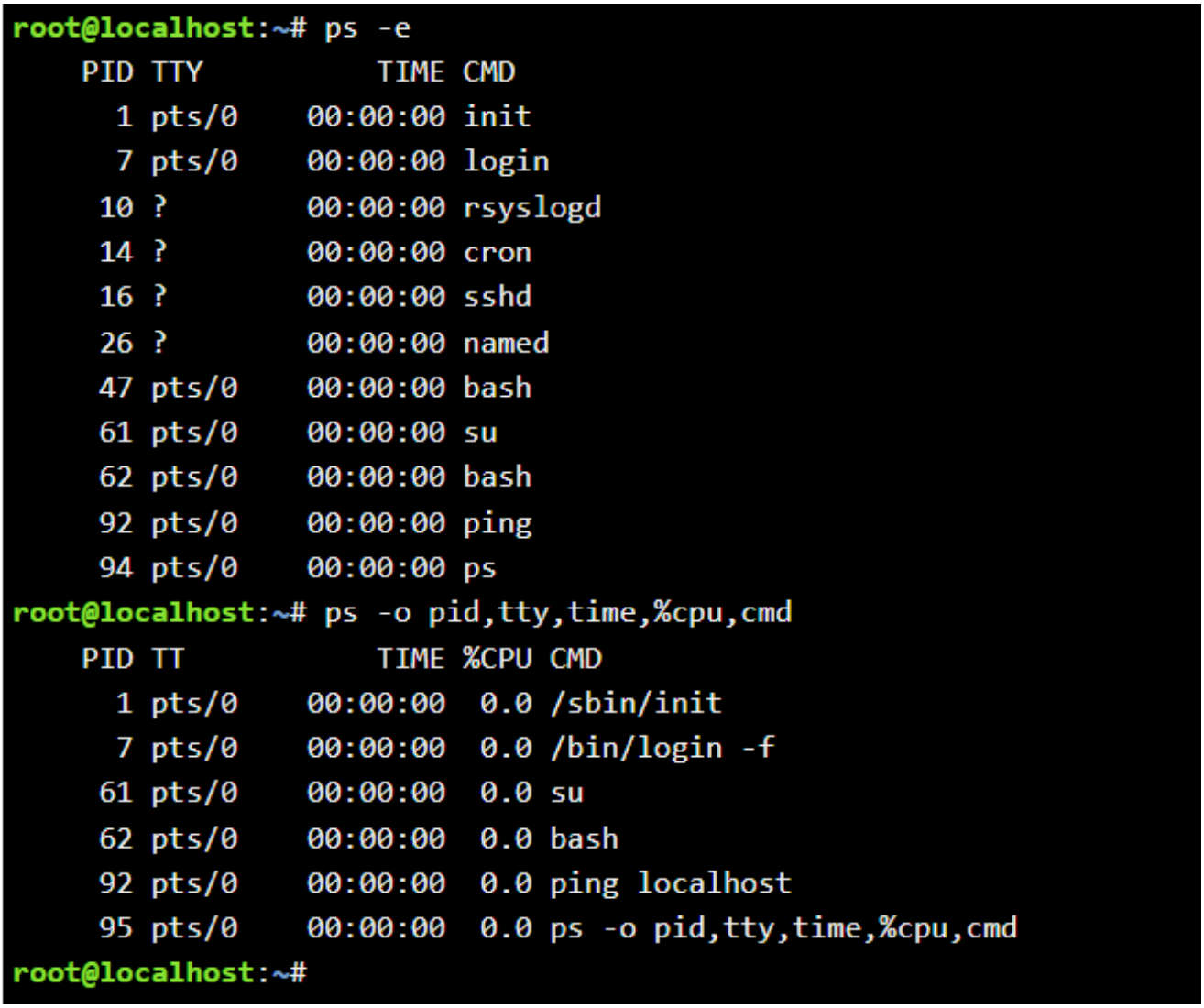

Due to this environment being a virtualized operating system, there are far fewer processes than what would normally be shown with Linux running directly on hardware.

# 13.6.3 Step 3

Use the `ps` command with the `-o` option to specify which columns to output.

```
ps -o pid,tty,time,%cpu,cmd
```

Your output should be similar to the following:



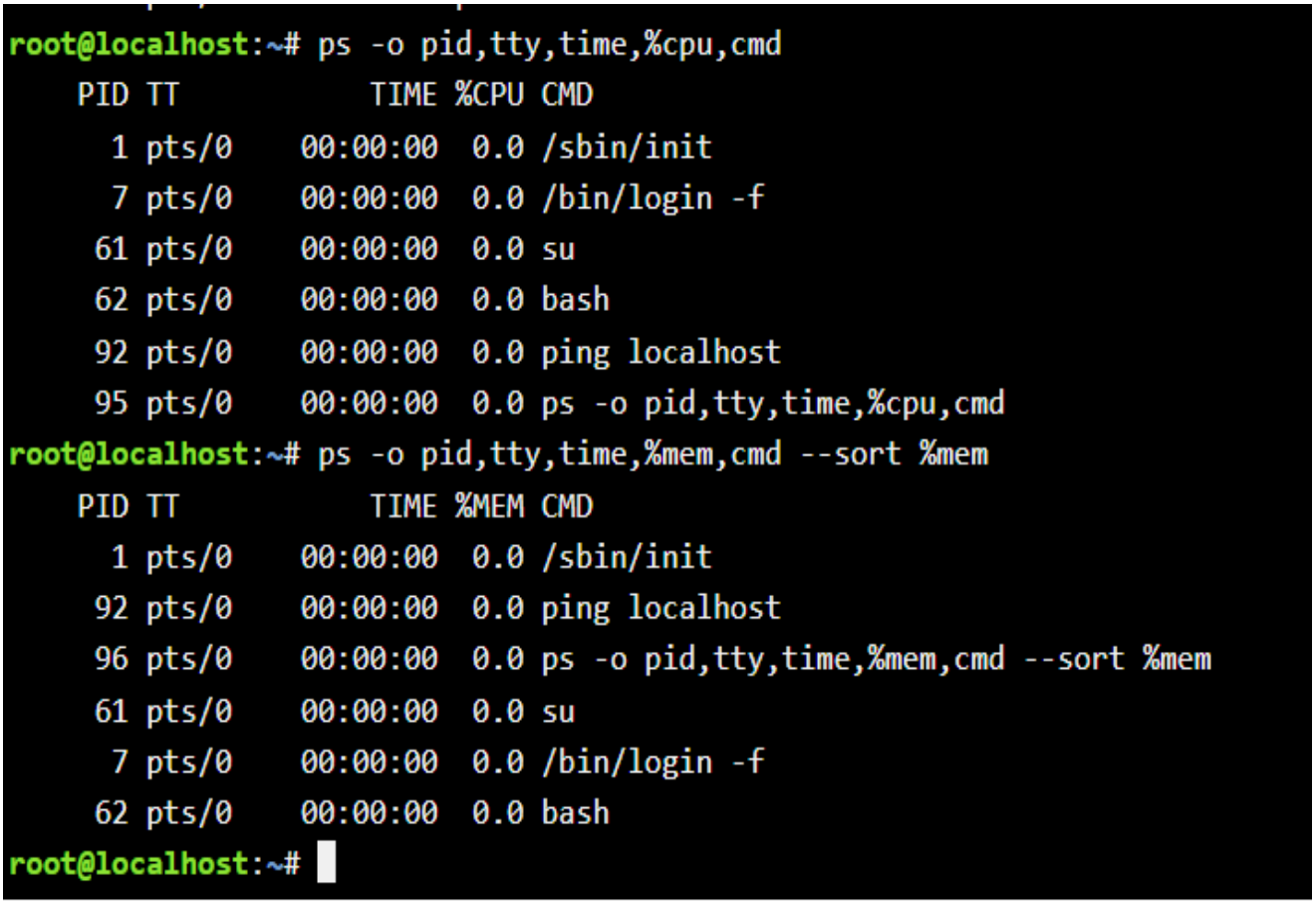
# 13.6.4 Step 4

Use the `--sort` option to specify which column(s) to sort by. By default, a column specified for sorting will be in ascending order, this can be forced with placing a plus `+` symbol in front of the column name. To specify a descending sort, use the minus `-` symbol in front of the column name.

Sort the output of `ps` by `%mem`:

```
ps -o pid,tty,time,%mem,cmd --sort %mem
```

Your output should be similar to the following:

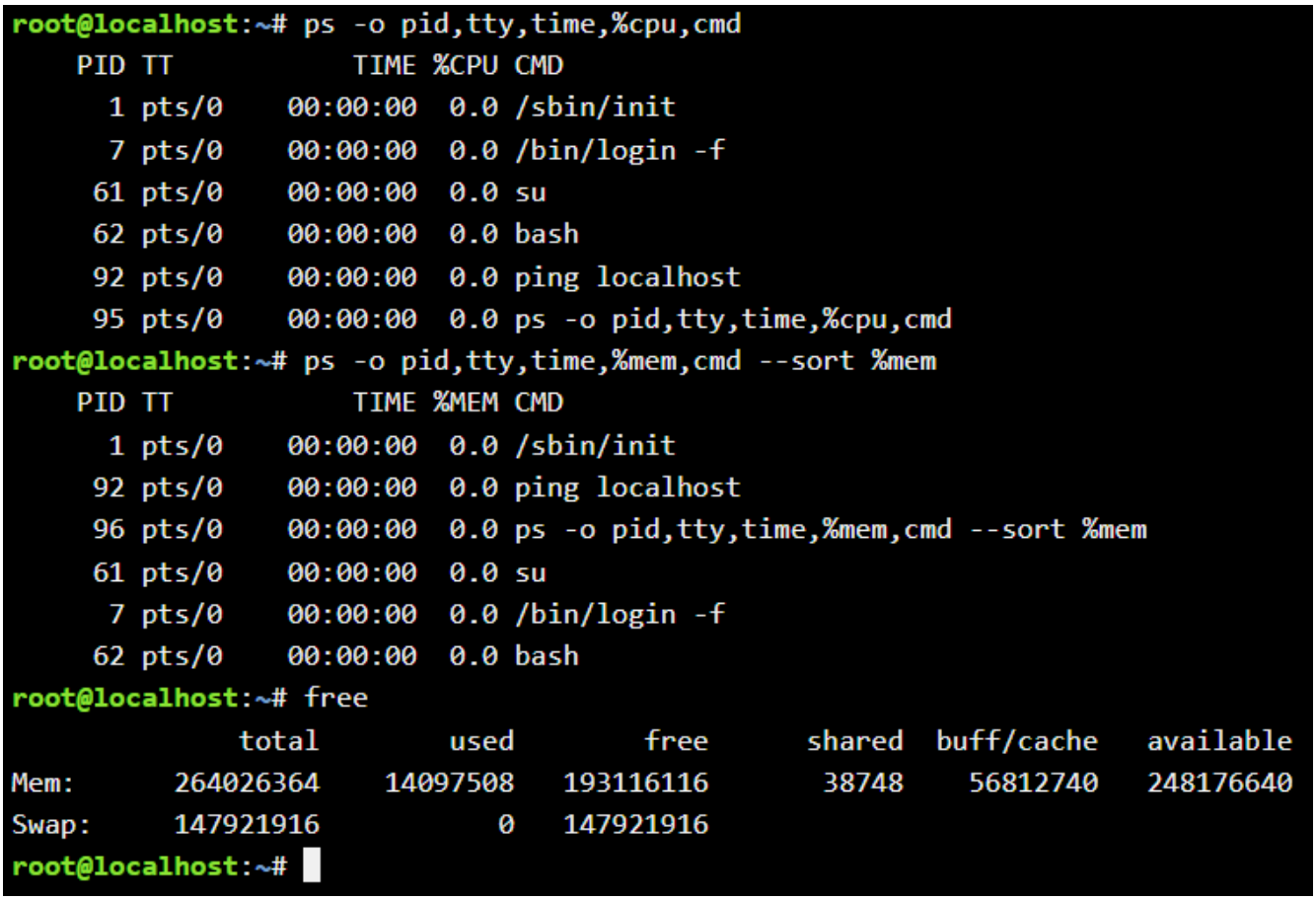


# 13.6.5 Step 5

While the `ps` command can show the percentage of memory that a process is using, the `free` command will show overall system memory usage:

```
free
```

Your output should be similar to the following:



# 13.6.6 Step 6

Stop the `ping` command with the following `kill` command and verify with the `jobs` command:

```
kill PID
jobs
```

Your output should be similar to the following:

```
root@localhost:~# ps -o pid,tty,time,%cpu,cmd
    PID TT           TIME %CPU CMD
      1 pts/0    00:00:00  0.0 /sbin/init
      7 pts/0    00:00:00  0.0 /bin/login -f
     61 pts/0    00:00:00  0.0 su
     62 pts/0    00:00:00  0.0 bash
     92 pts/0    00:00:00  0.0 ping localhost
     95 pts/0    00:00:00  0.0 ps -o pid,tty,time,%cpu,cmd
root@localhost:~# ps -o pid,tty,time,%mem,cmd --sort %mem
    PID TT           TIME %MEM CMD
      1 pts/0    00:00:00  0.0 /sbin/init
     92 pts/0    00:00:00  0.0 ping localhost
     96 pts/0    00:00:00  0.0 ps -o pid,tty,time,%mem,cmd --sort %mem
     61 pts/0    00:00:00  0.0 su
      7 pts/0    00:00:00  0.0 /bin/login -f
     62 pts/0    00:00:00  0.0 bash
root@localhost:~# free
              total        used        free      shared  buff/cache   available
Mem:      264026364    14097508   193116116       38748    56812740   248176640
Swap:     147921916           0   147921916
root@localhost:~# kill 92
root@localhost:~# jobs
[1]+  Terminated              ping localhost > /dev/null
root@localhost:~#
```

# 13.7 Viewing System Logs

System logs are critical for many tasks, including fixing operating system problems and ensuring that your system is secure. Knowing where the system log files are stored and how to maintain them is important for a system administrator.

There are two daemons that handle log messages: the `syslogd` daemon and the `klogd` daemon. Typically you don't need to worry about `klogd`; it only handles kernel log messages and it sends its log information to the `syslogd` daemon.

Messages that were generated by the kernel at boot time are stored in the `/var/log/dmesg` file. The `dmesg` command allows viewing of current kernel messages, as well as providing control of whether those messages will appear in a terminal console window.

The main log file that is written to by `syslogd` is `/var/log/`messages.

In addition to the logging done by `syslogd`, many other processes perform their own logging. Some examples of processes that do their own logging include the Apache web server (log file resides in the `/var/log/httpd` directory), the Common Unix Printing System (`/var/log/cups`) and the `auditd` daemon (`/var/log/audit`).

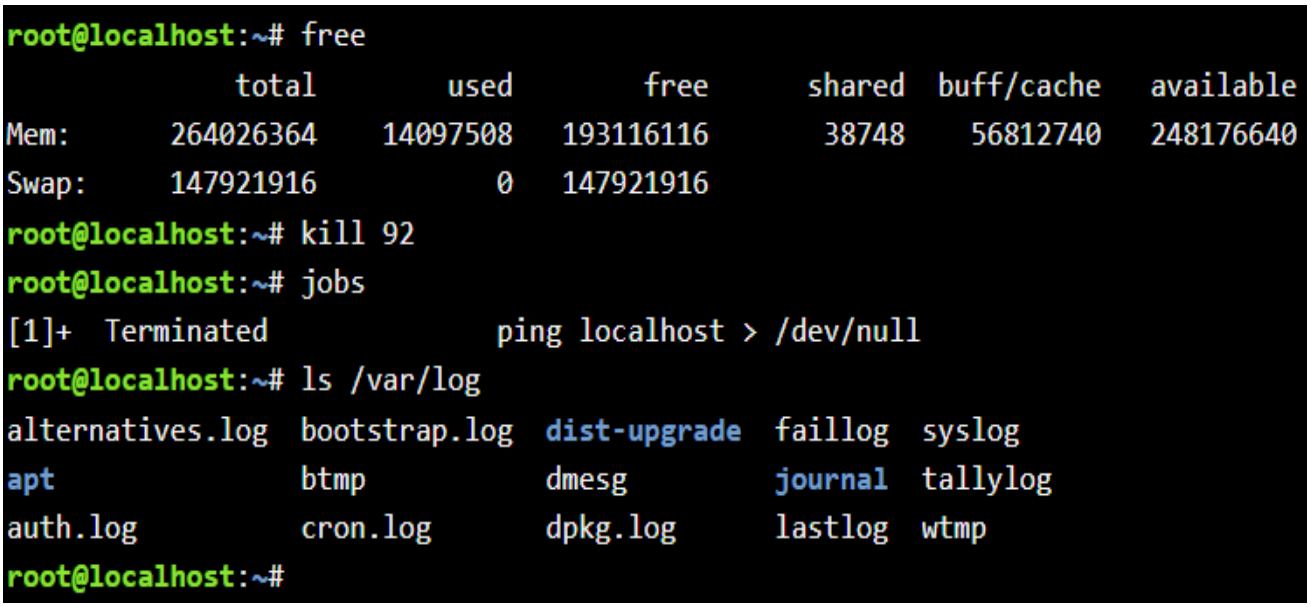
**Note**

On CentOS systems, the `syslogd` is called `rsyslogd`.

# 13.7.1 Step 1

System logs are stored in the `/var/log` directory. List the files in this directory:

```
ls /var/log
```



# 13.7.2 Step 2

Each log file represents a service or feature. For example, the `auth.log` file displays information regarding authorization or authentication, such as user login attempts. New data is stored at the bottom of the file. Execute the following commands to see an example:

```
ssh localhost
{At the first prompt, type yes}
{At the second prompt, type abc}
{At the third prompt, type abc}
{At the fourth prompt, type abc}
tail -5 /var/log/auth.log
```

```
root@localhost:~# ssh localhost
The authenticity of host 'localhost (::1)' can't be established.
ECDSA key fingerprint is SHA256:PSoO/gcBlkQxvtohpydgaSJlXLmNbHRRYJ77wauYcrE.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added 'localhost' (ECDSA) to the list of known hosts.
root@localhost's password:
Permission denied, please try again.
root@localhost's password:
Permission denied, please try again.
root@localhost's password:
root@localhost: Permission denied (publickey,password).
root@localhost:~# tail -5 /var/log/auth.log
Apr  8 19:20:47 localhost sshd[103]: pam_unix(sshd:auth): authentication failure
; logname= uid=0 euid=0 tty=ssh ruser= rhost=::1  user=root
Apr  8 19:20:49 localhost sshd[103]: Failed password for root from ::1 port 5266
8 ssh2
Apr  8 19:21:06 localhost sshd[103]: message repeated 2 times: [ Failed password
 for root from ::1 port 52668 ssh2]
Apr  8 19:21:06 localhost sshd[103]: Connection closed by authenticating user ro
ot ::1 port 52668 [preauth]
Apr  8 19:21:06 localhost sshd[103]: PAM 2 more authentication failures; logname
= uid=0 euid=0 tty=ssh ruser= rhost=::1  user=root
root@localhost:~#
```

The `ssh` command was used to generated data in the `/var/log/auth.log` file. Note the failed login attempts are logged in the `/var/log/auth.log` file.